EE627 — Class Summary

January 30, 2026

Prof. Rensheng Wang

Contents

1 XGBoost: What It Is and Why It Matters **3**
1.1 Definition . . . 3
1.2 When to Use It . . . 3
1.3 Why This Is an Interview Topic . . . 3

2 The Hiring Manager Analogy: Decision Tree to XGBoost **4**
2.1 Decision Tree — One Interviewer . . . 4
2.2 Bagging (Bootstrap Aggregating) — Interview Panel . . . 4
2.3 Random Forest — Randomized Panel . . . 4
2.4 Boosting — Sequential Interviews with Feedback . . . 4
2.5 Gradient Boosting — Measure the Gap with Calculus . . . 4
2.6 XGBoost — Gradient Boosting on Steroids . . . 5

3 Why XGBoost Performs So Well **6**
3.1 System Optimizations . . . 6
3.1.1 Parallelization . . . 6
3.1.2 Tree Pruning (Depth-First) . . . 6
3.1.3 Hardware Optimization (Cache Awareness) . . . 6
3.2 Algorithmic Enhancements . . . 6
3.2.1 Regularization (Built-In L1 + L2) . . . 6
3.2.2 Sparsity Awareness (Missing Value Handling) . . . 7
3.2.3 Weighted Quantile Sketch . . . 7
3.2.4 Built-In Cross-Validation . . . 7

4 Benchmark: Where Is the Proof? **8**

5 The "No Free Lunch" Caveat **9**
5.1 Key Hyperparameters to Know . . . 9
5.2 Python Code . . . 9

6 XGBoost vs. LightGBM **10**
6.1 What Is LightGBM? . . . 10
6.2 The Core Difference: Leaf-Wise vs. Level-Wise Tree Growth . . . 10
6.3 LightGBM Novel Techniques . . . 10
6.4 LightGBM Python Code . . . 10

6.5 Current Status . . . 11

7 Correlation: The Foundation of Time Series **12**
7.1 Why Correlation? . . . 12
7.2 Pearson Correlation Coefficient . . . 12
7.3 Computing Correlation in Python / MATLAB / R . . . 12

8 Time Series Analysis: Stationarity **13**
8.1 What Is a Time Series? . . . 13
8.2 Stationarity . . . 13

9 Auto-Correlation and the AR Model **14**
9.1 From Correlation to Auto-Correlation . . . 14
9.2 The Auto-Regressive (AR) Model . . . 14
9.3 Properties of the AR(1) Model . . . 14
9.4 Properties of the AR(2) Model . . . 15
9.5 The Lag Operator and ARMA (Reference) . . . 15

10 Missing Data: A Preview for Next Session **16**

11 Key Formulas Reference **17**

1 XGBoost: What It Is and Why It Matters

1.1 Definition

XGBoost (eXtreme Gradient Boosting) is a decision-tree-based *ensemble* machine learning algorithm that uses a gradient boosting framework.

- Open-source; core implemented in C++, with R/Python/Julia interfaces.
- Cloud-ready: supports AWS, Azure, Yarn; integrates with Flink and Spark.
- **The winning model for numerous Kaggle competitions.**

1.2 When to Use It

- **Structured (tabular) data** — CSV, TSV, SQL tables with named columns. XGBoost is the *first model to try* when you have no strong prior.
- **Not for unstructured data** (images, text). Those require deep learning (neural networks, transformers) because pixel or token positions carry no fixed semantic meaning that a tree can split on.

1.3 Why This Is an Interview Topic

XGBoost is a high-frequency data engineering interview question because:

1. It is not a yes/no question — it invites extended discussion.
2. Explaining *why* it works shows system-level understanding, not just API familiarity.
3. Knowing its limits (when *not* to use it) signals senior-level thinking.

2 The Hiring Manager Analogy: Decision Tree to XGBoost

The evolution of tree-based algorithms can be understood entirely through the lens of an iterative interview process.

2.1 Decision Tree — One Interviewer

A single hiring manager applies their own set of criteria: education level, years of experience, coding ability, communication. Each criterion is a *split* in the tree. Easy to visualize and explain (interpretable), but **biased** toward one person's perspective.

2.2 Bagging (Bootstrap Aggregating) — Interview Panel

Replace the single interviewer with a **committee**. Each member votes independently. The final decision aggregates all votes democratically.

- Reduces individual bias.
- Majority vote $\Rightarrow$ more robust, statistically reliable outcome.
- If 9 of 10 panelists approve: strong positive signal. Mixed votes: doubt.

2.3 Random Forest — Randomized Panel

Bagging, but each panelist only tests the candidate on a *randomly chosen subset* of criteria (e.g., one tests coding, another tests system design, another tests behavioral). This randomness:

- Prevents any single feature from dominating.
- Helps the ensemble generalize to new data (avoids overfitting).
- A few bad random subsets are harmless when many subsets are sampled.

2.4 Boosting — Sequential Interviews with Feedback

Interviewers run sequentially. Before each session, the next interviewer receives **feedback from the previous one** and sharpens their questions accordingly. Each round focuses on the *residual gap* — the parts the previous round did not resolve.

- Feedback loop $\Rightarrow$ progressively more targeted evaluation.
- The model "learns from its mistakes" round by round.

2.5 Gradient Boosting — Measure the Gap with Calculus

Boosting, but each round measures the residual gap using the **gradient** of the loss function — i.e., how far the current prediction is from the target, and in which direction to push.

Analogy from lecture: A candidate is great at communication but terrible at coding. The gradient tells you exactly *how large* that coding gap is. The next interview round focuses precisely there.

2.6 XGBoost — Gradient Boosting on Steroids

XGBoost improves on gradient boosting by also measuring the **Hessian** — the second derivative of the loss — telling the model *how quickly the gradient itself is changing.*

- First derivative (gradient g_i): how far off the prediction is.
- Second derivative (Hessian h_i): how fast that error is changing — how aggressively to step toward the minimum.
- Combined, these enable fewer iterations and faster convergence than standard gradient boosting.

This is combined with state-of-the-art software and hardware optimizations (see Section 3).

3 Why XGBoost Performs So Well

XGBoost improves on standard Gradient Boosting Machines (GBMs) through **system optimizations** and **algorithmic enhancements**.

3.1 System Optimizations

3.1.1 Parallelization

Standard tree building is sequential (outer loop over leaf nodes, inner loop over features). XGBoost swaps this loop order, enabling features to be evaluated simultaneously across parallel threads.

Why it matters: From 100 features, choosing a random subset of 20 yields a combination space larger than 10^{15}. Sequential evaluation would be computationally infeasible. Parallelization makes it tractable.

3.1.2 Tree Pruning (Depth-First)

Standard GBMs use a greedy stopping criterion: stop splitting when the split would be negative. This is "short-sighted" — a seemingly bad split now might enable a much better split one level deeper.

XGBoost uses `max_depth` and grows trees **depth-first**, then *prunes backward*. This finds globally better tree structures.

3.1.3 Hardware Optimization (Cache Awareness)

Each thread allocates an internal buffer to store gradient statistics locally. This avoids redundant memory accesses — values computed once are cached and reused.

Connection: The same cache-awareness principle drives the key-value (KV) cache in Transformer models, where intermediate attention computations are stored and reused rather than recalculated.

3.2 Algorithmic Enhancements

3.2.1 Regularization (Built-In L1 + L2)

XGBoost's objective penalizes model complexity with *both* L1 (Lasso) and L2 (Ridge) regularization simultaneously:

$$\mathcal{L}(\phi) = \sum_i l(\hat{y}_i,\, y_i) + \sum_k \Omega(f_k), \quad \Omega(f) = \gamma T + \tfrac{1}{2}\lambda \|w\|^2$$

where T is the number of leaves and w are the leaf weights. You do *not* need to add a regularizer manually — it is baked in.

L1 vs. L2 (a frequent interview question):

- **L1 (Lasso, `alpha`):** penalizes $|w_j|$. Geometrically, the constraint region has corners, which forces many weights to exactly zero $\Rightarrow$ **fast, aggressive feature elimination**. Use when you want dimension reduction.
- **L2 (Ridge, `lambda`):** penalizes w_j^2. Constraint region is smooth; weights shrink toward zero but rarely hit it exactly $\Rightarrow$ **slower, gentler shrinkage**. Keeps all features.

3.2.2 Sparsity Awareness (Missing Value Handling)

When a value is missing in the feature matrix, XGBoost learns the optimal *default direction* (left or right branch) from the training data itself. The algorithm tries both directions and keeps whichever yields better performance.

Practical result: You do *not* need to impute missing values before running XGBoost. The model handles them natively and intelligently.

Contrast with legacy tools: MATLAB and older Python libraries returned errors on missing data, forcing analysts to delete rows or manually impute values (mean, zero, or custom). Both choices introduce bias. XGBoost eliminates this pain.

3.2.3 Weighted Quantile Sketch

Before building any tree, XGBoost pre-scans the data to compute *percentile breakpoints* (quantiles) for each feature. Tree splits are proposed only at these breakpoints, not at arbitrary values.

Why: A split at age = 10 is meaningless if everyone in the dataset is over 18. By understanding the data distribution first, every proposed split is statistically defensible and the same quantile information can be reused across multiple trees without recomputation.

3.2.4 Built-In Cross-Validation

XGBoost performs internal train/validation splitting at each boosting iteration. You do not need to manually code a 70/30 split or k-fold cross-validation — it is already happening inside the model.

4 Benchmark: Where Is the Proof?

Using scikit-learn's `Make_Classification` dataset (1 million points, 20 features, 5-fold CV):

Algorithm	**AUROC**	**Training Time (s)**
XGBoost	0.9662	**24**
Gradient Boosting	0.9661	2,069
Random Forest	0.9542	424
Logistic Regression	0.9373	17

XGBoost matches the best accuracy while being **86× faster** than standard Gradient Boosting.

5 The "No Free Lunch" Caveat

XGBoost is the default starting point, not a guaranteed winner. Real practice requires:

1. Testing multiple algorithms on your specific data.
2. **Hyperparameter tuning** — a model with default settings is never fully optimized.
3. Considering computational complexity, explainability requirements, and ease of deployment.

From the slides: "This is exactly the point where Machine Learning starts drifting away from science towards art — but honestly, that's where the magic happens."

5.1 Key Hyperparameters to Know

Parameter	Controls	Typical Range
`learning_rate`	Step size per boosting round	0.01 – 0.3
`n_estimators`	Number of boosting rounds (trees)	100 – 1,000
`max_depth`	Maximum depth of each tree	3 – 10
`min_child_weight`	Min sum of instance weight to split further	1 – 10
`subsample`	Fraction of rows sampled per tree	0.5 – 1.0
`colsample_bytree`	Fraction of columns sampled per tree	0.5 – 1.0
`lambda`	L2 regularization weight	0 – 10
`alpha`	L1 regularization weight (Lasso)	0 – 1
`gamma`	Min loss reduction required to make a split	0 – 5

Practical advice from lecture: Most parameters will show little sensitivity on a given dataset. The value of tuning is to *find the one or two that are sensitive* — those are the parameters that reveal the structure of your specific data.

5.2 Python Code

```
# Regression
from xgboost import XGBRegressor
from sklearn.model_selection import cross_val_score
scores = cross_val_score(
    XGBRegressor(objective='reg:squarederror'),
    X, y, scoring='neg_mean_squared_error')

# Classification
from xgboost import XGBClassifier
cross_val_score(XGBClassifier(), X, y)
```

6 XGBoost vs. LightGBM

6.1 What Is LightGBM?

Microsoft's **LightGBM** (Light Gradient Boosting Machine) is a more efficient variant of gradient boosting, now preferred by many Kagglers.

- Up to **6**× faster than XGBoost.
- Lower memory usage.
- Supports parallel and GPU learning.
- Capable of handling very large-scale data.

6.2 The Core Difference: Leaf-Wise vs. Level-Wise Tree Growth

Property	**XGBoost (Level-Wise)**	**LightGBM (Leaf-Wise)**
Growth strategy	Scan level by level (BFS)	Always expand highest-loss leaf (DFS)
Speed	Slower	Faster
Accuracy (small trees)	Comparable	Often better
Accuracy (full tree)	Same (converge to same tree)	Same
Monotonic constraints	Supported	Not supported
Interpretability	Easier	Slightly harder

Key insight: Leaf-wise growth chooses splits based on their contribution to *global* loss, not just the loss along one branch. For a fixed node budget, it often produces lower-error trees than level-wise growth.

6.3 LightGBM Novel Techniques

- **GOSS (Gradient-based One-Side Sampling):** Drops data instances with small gradients (they are already well-predicted). Keeps large-gradient instances and a random sample of small-gradient ones. Accurate information gain estimate at a fraction of the data size.
- **EFB (Exclusive Feature Bundling):** Bundles mutually exclusive features (features that are rarely non-zero simultaneously) into a single feature. Reduces the effective number of features, speeding up split finding.

6.4 LightGBM Python Code

```
from sklearn.model_selection import train_test_split
import lightgbm as lgb

X_train, X_test, y_train, y_test = \
    train_test_split(X, y, test_size=0.3, random_state=0)

clf = lgb.LGBMClassifier()
clf.fit(X_train, y_train)
```

6.5 Current Status

XGBoost developers later added `grow_policy='lossguide'` to match LightGBM's leaf-wise behavior. Even so, LightGBM remains approximately 1.3×–1.5× faster than XGBoost. Both are viable; XGBoost's monotonic constraint support can be valuable when model interpretability is a business requirement.

7 Correlation: The Foundation of Time Series

7.1 Why Correlation?

Correlation is the most fundamental measurement used throughout time series analysis. Its applications are:

- **Prediction:** if X and Y are correlated, knowing one lets you predict the other.
- **Feature selection:** highly correlated features are redundant; low-correlation features with the target are uninformative.
- **Reliability / validity:** measuring whether two instruments or raters agree.

7.2 Pearson Correlation Coefficient

The correlation coefficient between X and Y is:

$$r_{x,y} = \frac{\mathrm{Cov}(X,Y)}{\sqrt{\mathrm{Var}(X)\,\mathrm{Var}(Y)}} = \frac{E[(X-\mu_x)(Y-\mu_y)]}{\sqrt{E(X-\mu_x)^2\,E(Y-\mu_y)^2}}$$

- $-1 \leq r_{x,y} \leq 1$ always.
- $r \approx +1$: strong positive linear relationship.
- $r \approx -1$: strong negative linear relationship.
- $r \approx 0$: little or no *linear* association.

Critical subtlety: $r = 0$ means *uncorrelated*, **not** *independent*. Two variables can have a strong non-linear relationship (e.g., $Y = X^2$) and still have $r = 0$.

7.3 Computing Correlation in Python / MATLAB / R

```
# Python (NumPy)
import numpy as np
r = np.corrcoef(x, y)[0, 1]

# MATLAB
r = corrcoef(A)          % correlation matrix of all columns

% R
cor(x, y)                # scalar; cor(A) gives full matrix
```

8 Time Series Analysis: Stationarity

8.1 What Is a Time Series?

A time series is data indexed by a sequential, **non-swappable** time index. Order matters: shuffling destroys the dependency structure.

The same logic applies to any ordered sequence:

- Financial prices (daily/hourly returns)
- Text tokens (word order in a sentence)
- Geospatial measurements (neighboring locations)
- Audio/spectral data (frequency bins)

This is why understanding time series transfers directly to understanding language models and diffusion models.

8.2 Stationarity

The foundation of time series analysis. We need the process to be *stable over time* so that past observations predict future ones.

Strict stationarity: The joint distribution of $\{X_{t_1}, \ldots, X_{t_k}\}$ is identical to that of $\{X_{t_1+l}, \ldots, X_{t_k+l}\}$ for all l. Rarely verified in practice.

Weak (2nd-order) stationarity: The practical version. $\{X_t\}$ is weakly stationary if both:

- The mean $E(X_t) = \mu$ is constant (does not depend on t).
- The covariance $\mathrm{Cov}(X_t, X_{t-l}) = v_l$ depends only on the *lag* l, not on t.

Examples of non-stationary processes:

- **Structural break:** the mean shifts at some time k (e.g., $y_t = \beta + e_t$ for $t \leq k$, $y_t = \beta + \lambda + e_t$ for $t > k$).
- **Random walk (unit root):** $y_t = y_{t-1} + e_t$. Variance grows without bound: $\mathrm{Var}(y_t) > \mathrm{Var}(y_{t-1})$, so it is not stationary.

9 Auto-Correlation and the AR Model

9.1 From Correlation to Auto-Correlation

Apply the correlation formula with $X \to X_t$ and $Y \to X_{t-l}$ (the same series shifted by lag l). The result is **auto-correlation**.

Define the auto-covariance at lag l:

$$v_l = \mathrm{Cov}(X_t,\, X_{t-l}) = E[(X_t - \mu)(X_{t-l} - \mu)]$$

The **auto-correlation function (ACF)** is the normalized version:

$$r_l = \frac{v_l}{v_0}, \quad r_0 = 1, \quad |r_l| \le 1, \quad r_l = r_{-l}$$

where $v_0 = \mathrm{Var}(X_t)$.

What r_l means: The correlation r_l between X_t and X_{t-l} quantifies how much knowing X_{t-l} reduces uncertainty about X_t, without any other information. The best linear predictor of X_t from X_{t-l} is:

$$X_t - \mu \;\approx\; r_l\,(X_{t-l} - \mu)$$

Practical formula for a time series of length N:

$$\mathrm{ACF}[l] = \frac{1}{N-l}\sum_{k=1}^{N-l}(x[k] - m)(x[k+l] - m)$$

Note the denominator is $N - l$ (not N) because only $N - l$ overlapping samples exist after shifting by l.

9.2 The Auto-Regressive (AR) Model

If a stationary time series can be explained as a linear combination of its own past values plus noise, it is called an **autoregressive model of order** p, written AR(p):

$$X_t = a_0 + a_1 X_{t-1} + a_2 X_{t-2} + \cdots + a_p X_{t-p} + \varepsilon_t$$

where ε_t is a zero-mean random noise term and a_i are coefficients.

9.3 Properties of the AR(1) Model

The simplest case, AR(1): $X_t = a_0 + a_1 X_{t-1} + \varepsilon_t$.

- **Mean:** $E(X_t) = \mu = \dfrac{a_0}{1 - a_1}$ (exists only when $a_1 \neq 1$).
- **Variance:** $\mathrm{Var}(X_t) = v_0 = \dfrac{\sigma_\varepsilon^2}{1 - a_1^2}$
- **Stationarity condition:** $|a_1| < 1$. If $|a_1| \ge 1$, the variance is infinite or negative — the series explodes or is undefined.

- **ACF decay:** $r_l = a_1^l$. The autocorrelation of a stationary AR(1) series decays **exponentially** with rate a_1.

Intuition: If $a_1 = 0.9$, yesterday's value explains 90% of today's fluctuation around the mean. If $a_1 = 0.1$, the series is nearly memoryless — past values barely matter.

9.4 Properties of the AR(2) Model

AR(2): $X_t = a_0 + a_1 X_{t-1} + a_2 X_{t-2} + \varepsilon_t$.

- **Mean:** $\mu = \dfrac{a_0}{1 - a_1 - a_2}$ (requires $a_1 + a_2 \neq 1$).
- **ACF recursion:** $r_l = a_1 r_{l-1} + a_2 r_{l-2}$ for $l \geq 2$.
- **Lag-1 ACF:** $r_1 = \dfrac{a_1}{1 - a_2}$
- The ACF of a stationary AR(2) satisfies the second-order difference equation $(1 - a_1 B - a_2 B^2)\, r_l = 0$, where B is the back-shift operator ($B r_l = r_{l-1}$).

9.5 The Lag Operator and ARMA (Reference)

The **lag operator** L is defined by $L X_t = X_{t-1}$. Using it, AR(p) becomes $a(L) X_t = \varepsilon_t$ where $a(L) = 1 - a_1 L - \cdots - a_p L^p$.

An **ARMA(p, q)** model combines autoregressive and moving-average terms:

$$a(L)\, y_t = b(L)\, e_t$$

where $b(L) = 1 + \theta_1 L + \cdots + \theta_q L^q$. This framework unifies AR and MA models and forms the backbone of classical time series forecasting.

10 Missing Data: A Preview for Next Session

The transcript closed with a preview of the *missing data* discussion coming next week. The key point is that the right answer is always **"it depends"**, not a single prescribed action. The questions to ask are:

1. **What fraction is missing?** — 0.1% vs. 30% call for very different treatments.
2. **Which variables are missing?** — Categorical or continuous? High-importance or low-importance features?
3. **How are they missing?** — Missing in a contiguous range? Missing correlated with another variable? Independently at random?

Giving this structured answer in an interview signals that you understand data qualitatively, not just algorithmically. XGBoost handles missing values natively (Section 3), but understanding *why* the data is missing is still the analyst's job.

11 Key Formulas Reference

Pearson Correlation: $r_{x,y} = \dfrac{\mathrm{Cov}(X,Y)}{\sqrt{\mathrm{Var}(X)\,\mathrm{Var}(Y)}}, \quad -1 \le r \le 1$

Auto-Covariance (lag l, length N): $\mathrm{ACF}[l] = \dfrac{1}{N-l}\displaystyle\sum_{k=1}^{N-l}(x[k]-m)(x[k+l]-m)$

Auto-Correlation: $r[l] = \dfrac{\mathrm{ACF}[l]}{\mathrm{ACF}[0]}, \quad r[0]=1, \quad |r[l]| \le 1$

AR(p) Model: $X_t = a_0 + a_1 X_{t-1} + \cdots + a_p X_{t-p} + \varepsilon_t$

AR(1) Mean: $\mu = \dfrac{a_0}{1-a_1}$

AR(1) Variance: $\mathrm{Var}(X_t) = \dfrac{\sigma_\varepsilon^2}{1-a_1^2}, \quad$ requires $|a_1| < 1$

AR(1) ACF Decay: $r_l = a_1^l \quad$ (exponential decay)

XGBoost Objective: $\mathcal{L}(\phi) = \displaystyle\sum_i l(\hat{y}_i, y_i) + \sum_k \Omega(f_k), \quad \Omega(f) = \gamma T + \tfrac{1}{2}\lambda\|w\|^2$

Gradient Tree Boosting (2nd-order approximation): $\mathcal{L}^{(t)} \approx \displaystyle\sum_{i=1}^{n}\left[g_i f_t(\mathbf{x}_i) + \tfrac{1}{2}h_i f_t^2(\mathbf{x}_i)\right] + \Omega(f_t)$

where $g_i = \partial_{\hat{y}^{(t-1)}} l(y_i, \hat{y}^{(t-1)})$ and $h_i = \partial^2_{\hat{y}^{(t-1)}} l(y_i, \hat{y}^{(t-1)})$